

SKILL-5

Session-9

Task-1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>
#include <errno.h>

int main()
{
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX];

    printf("Directory Navigation Test\n");
    printf("-----\n");

    /* Get initial working directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(previous_dir, current_dir);

    printf("Initial directory : %s\n", current_dir);

    /* Change directory */
```

```
GNU nano 7.2 skill5s9task1.c *
{
    if (getcwd(current_dir, sizeof(current_dir)) != NULL)
        printf("Current directory : %s\n", current_dir);
    }
else
{
    perror("chdir");
}

/* Validate path */
printf("\nValidating path /tmp...\n");

if (access("/tmp", F_OK) == 0)
    printf("Path validation : /tmp exists\n");
else
    printf("Path validation : /tmp does not exist\n");

/* Handle invalid path */
printf("\nTesting invalid path...\n");

if (chdir("/invalid_directory") != 0)
    printf("Error handled : %s\n", strerror(errno));

/* Return to previous directory */
printf("\nReturning to previous directory...\n");
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/ Go To Line

```

/home/ruhi0404/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill5s9task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill5s9task1.c -o skill5s9task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill5s9task1
Directory Navigation Test
-----
Initial directory   : /home/ruhi0404/Operating_Systems

Changing directory to /tmp...
Current directory  : /tmp

Validating path /tmp...
Path validation    : /tmp exists

Testing invalid path...
Error handled      : No such file or directory

Returning to previous directory...
Current directory  : /home/ruhi0404/Operating_Systems

Navigation test completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully demonstrates directory navigation using `chdir()` and obtains the current working directory using `getcwd()`. The existence of a path is validated using `access()`. An invalid directory is tested and the resulting error is handled using `errno` and `strerror()`. The previous working directory is stored before navigation and restored after the test. Thus, directory changing, path validation, error handling, previous-directory maintenance, and navigation testing are successfully demonstrated.

Task-2: TO Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100

/* Function declarations */
void command_pwd();
void command_cd(char *path);
void command_help();
void command_exit();

/* Dispatch table structure */
struct Command
{
    char *name;
    void (*function)();
};

/* Built-in command dispatch table */
struct Command commands[] =
{
    {"pwd", command_pwd},
    {"help", command_help},
    {"exit", command_exit}
};

int command_count = 3;

/* Built-in command implementations */
void command_pwd()
{
    char cwd[1024];

```

^G Help

^O Write Out

^W Where Is

^K

```

        perror("pwd");
    }

void command_cd(char *path)
{
    if (path == NULL)
    {
        printf("cd: path required\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
}

void command_help()
{
    printf("Built-in commands:\n");
    printf("  pwd  - Display current directory\n");
    printf("  cd   - Change directory\n");
    printf("  help - Display available commands\n");
    printf("  exit - Exit the shell\n");
}

void command_exit()
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Execute external Linux commands */
void execute_external(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {

```

^G Help
^Y Exit

^O Write Out
^P Read File

^W Where Is
^_ Replace

^K
^U

```

    if (pid == 0)
    {
        execlp(command, command, NULL);

        perror("Command execution failed");
        exit(1);
    }
    else
    {
        wait(NULL);
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("Mini Command Dispatcher\n");
    printf("-----\n");
    printf("Type 'help' to see available commands.\n");

    while (1)
    {
        printf("\nmysHELL> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        /* Handle cd separately because it must change the shell's state */
        if (strncmp(input, "cd ", 3) == 0)
        {

```

```
    int found = 0;

    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(input, commands[i].name) == 0)
        {
            commands[i].function();
            found = 1;
            break;
        }
    }

    /* Execute external command */
    if (!found)
    {
        if (strcmp(input, "ls") == 0)
        {
            execute_external("ls");
            found = 1;
        }
        else if (strcmp(input, "date") == 0)
        {
            execute_external("date");
            found = 1;
        }
    }

    /* Invalid command */
    if (!found)
    {
        printf("Invalid command: %s\n", input);
    }
}

return 0;
```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill5s9task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill5s9task2.c -o skill5s9task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill5s9task2
Mini Command Dispatcher
-----
Type 'help' to see available commands.

myshell> help
Built-in commands:
pwd    - Display current directory
cd     - Change directory
help   - Display available commands
exit   - Exit the shell

myshell> pwd
Current directory: /home/ruhi0404/Operating_Systems

myshell> cd
cd: path required

myshell> ls
File1.txt include pra3task2.c pra5task2.c prac7task2.c skill2task1.c skill4s7task1.c skills3task2 skills5task1 skills8ttask2
File2.txt obj pra4task1.c prac1task1.c practical skill2task2.c skill4s7task2.c skills3task2.c skills5task1.c skills8ttask2.c
LICENSE pral1task1.c pra4task1.c prac1task1.c sample.txt skill2task2.c skill4s8task1.c skills3week2.c skills5task2.c skillw3task2.c
Makefile pral1task1.c pra4task1.c prac2task1.c scripts skill2w1task1.c skill4s8task1.c skills3week2.c skills5task2.c skillw3task2.c
README.md pra2task2.c pra4task2.c prac2task1.c skill skill2w1task1.c skill4s9task1.c skills4task1.c skills6task1.c src
assets pra3task1.c pra5task1.c prac7task1.c skill1task2 skill2w1task2.c skill4s9task1.c skills4task1.c skills6task1.c tests
bin pra3task1.c pra5task1.c prac7task1.c skill1task2.c skill2w1task2.c skill4s9task2.c skills4task2.c skills6task2.c
docs pra3task2.c pra5task2.c prac7task2.c skill2task1 skill4s7task1.c skill4s9task2.c skills4task2.c skills6task2.c

myshell> date
Fri Sep 18 03:02:45 UTC 2026

myshell> hello
Invalid command: hello

myshell> exit
Exiting shell...
ruhi0404@Ruhitha:~/Operating_Systems$

```

Observation

The program successfully implements a command dispatcher using a dispatch table containing built-in commands. Commands such as `pwd`, `help`, `cd`, and `exit` are identified and processed according to their corresponding functions. External Linux commands such as `ls` and `date` are executed using `fork()` and `exec1p()`. Invalid commands are detected and an appropriate error message is displayed. The `cd` command maintains the shell's current working directory, demonstrating state maintenance. The dispatch logic was successfully tested using valid, external, and invalid commands.

SESSION-10

Task-1: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>

int main()
{
    char current_dir[PATH_MAX];
    FILE *state_file;

    printf("Shell State and Exit Handling\n");
    printf("-----\n");

    /* Retrieve current working directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    /* Display current path */
    printf("Current directory: %s\n", current_dir);

    /* Save current directory as state */
    state_file = fopen("shell_state.txt", "w");

    if (state_file == NULL)
    {
        perror("fopen");
        return 1;
    }

    fprintf(state_file, "%s\n", current_dir);

    printf("State saved successfully.\n");

    /* Cleanup resource */
```



```

/* Retrieve current working directory */
if (getcwd(current_dir, sizeof(current_dir)) == NULL)
{
    perror("getcwd");
    return 1;
}

/* Display current path */
printf("Current directory: %s\n", current_dir);

/* Save current directory as state */
state_file = fopen("shell_state.txt", "w");

if (state_file == NULL)
{
    perror("fopen");
    return 1;
}

fprintf(state_file, "%s\n", current_dir);

printf("State saved successfully.\n");

/* Cleanup resource */
fclose(state_file);

printf("State file closed successfully.\n");

/* Process exit request */
printf("Exit request received.\n");
printf("Cleaning up resources...\n");

printf("Program exiting successfully.\n");

return 0;
}

```

⌘G Help
⌘O Write Out
⌘W Where Is
⌘K Cut
⌘T Execute

```

Exiting Shell...
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill5s10task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill5s10task1.c -o skill5s10task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill5s10task1
Shell State and Exit Handling
-----
Current directory: /home/ruhi0404/Operating_Systems
State saved successfully.
State file closed successfully.
Exit request received.
Cleaning up resources...
Program exiting successfully.
ruhi0404@Ruhitha:~/Operating_Systems$ cat shell_state.txt
/home/ruhi0404/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ ls
File1.txt      obj          pra4task1.c  prac2task1  shell_state.txt  skill2w1task1.c  skill5s10task1.c  skills4task1  skills6task1.c
File2.txt      pral1task1.c pra4task2.c  prac2task1.c skill          skill2w1task2.c  skill5s9task1.c  skills4task1.c skills6task2.c
LICENSE        pra2task1.c  pra4task2.c  prac7task1.c skill1task2.c  skill4s7task1.c  skill5s9task2.c  skills4task2.c skills6ttask2.c
Makefile       pra3task1.c  pra5task1.c  prac7task2.c skill2task1.c  skill4s7task1.c  skill5s9task2.c  skills5task1.c skills8ttask2.c
README.md      pra3task1.c  pra5task2.c  prac7task2.c skill2task1.c  skill4s7task2.c  skill5s9task2.c  skills5task1.c skills8ttask2.c
assets         pra3task2.c  pra5task2.c  practical   skill2task2.c  skill4s8task1.c  skill5s3task2.c  skills5task2.c skillw3task2.c
bin            pra3task2.c  pra5task2.c  sample.txt  skill2task2.c  skill4s8task1.c  skill5s3task2.c  skills5task2.c skillw3task2.c
docs           pra3task2.c  pra5task2.c  scripts     skill2w1task1  skill5s10task1  skill5s3week2.c  skills6task1  tests
include        pra4task1    pracltask1.c scripts     skill2w1task1  skill5s10task1  skill5s3week2.c  skills6task1  tests
ruhi0404@Ruhitha:~/Operating_Systems$

```

Observation

The program successfully retrieves and displays the current working directory using `getcwd()`. The current directory path is saved into a state file using file operations. The opened file is properly closed using `fclose()` to release the resource. The program then processes the exit request and terminates normally. The saved state is verified by reading the contents of `shell_state.txt`.

Task-2: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 200

/* Validate environment variable name */
int valid_name(const char *name)
{
    int i;

    if (name[0] == '\\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (i = 1; name[i] != '\\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

/* Process export command */
void process_export(char *input)
{
    char *equal;
    char name[MAX_INPUT];
    char *value;

    equal = strchr(input, '=');
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^\\ Replace

^K Cut
^U Paste

```

    {
        printf("Invalid export syntax. Use: export NAME=value\n");
        return;
    }

    /* Extract variable name */
    int name_length = equal - input;

    if (name_length <= 0 || name_length >= MAX_INPUT)
    {
        printf("Invalid variable name.\n");
        return;
    }

    strncpy(name, input, name_length);
    name[name_length] = '\0';

    value = equal + 1;

    /* Validate variable name */
    if (!valid_name(name))
    {
        printf("Invalid variable name: %s\n", name);
        return;
    }

    /* Check whether variable already exists */
    if (getenv(name) != NULL)
    {
        printf("Existing variable '%s' found. Updating value.\n", name);
    }
    else
    {
        printf("Creating new variable '%s'.\n", name);
    }

    /* Update environment */

```

```

    {
        char *value = getenv(name);
        if (value != NULL)
            printf("Child process: %s=%s\n", name, value);
        else
            printf("Child process: variable not found.\n");

        exit(0);
    }
    else
    {
        wait(NULL);
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("Environment Variable Export Test\n");
    printf("-----\n");
    printf("Use: export NAME=value\n");
    printf("Use: test NAME to check child process.\n");
    printf("Use: exit to quit.\n");

    while (1)
    {
        printf("\nmyshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;
    }
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

 26°C
 Partly sunny



 Search



```
while (1)
{
    printf("\nmyshell> ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting...\n");
        break;
    }

    if (strncmp(input, "export ", 7) == 0)
    {
        process_export(input + 7);
    }
    else if (strncmp(input, "test ", 5) == 0)
    {
        test_child_process(input + 5);
    }
    else if (strlen(input) == 0)
    {
        continue;
    }
    else
    {
        printf("Invalid command.\n");
    }
}

return 0;
```

```
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill5s10task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill5s10task2.c -o skill5s10task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill5s10task2
Environment Variable Export Test
-----
Use: export NAME=value
Use: test NAME to check child process.
Use: exit to quit.

myshell> export MYNAME=Ruhitha
Creating new variable 'MYNAME'.
Export successful: MYNAME=Ruhitha

myshell> test MYNAME
Child process: MYNAME=Ruhitha

myshell> export MYNAME=Student
Existing variable 'MYNAME' found. Updating value.
Export successful: MYNAME=Student

myshell> test MYNAME
Child process: MYNAME=Student

myshell> export 123ABC=hello
Invalid variable name: 123ABC

myshell> export HELLO
Invalid export syntax. Use: export NAME=value

myshell> exit
Exiting...
ruhi0404@Ruhitha:~/Operating_Systems$ |
```

Observation

The program successfully parses the `export NAME=value` syntax and separates the variable name from its value. The variable name is validated according to standard environment-variable naming rules. The `setenv()` function is used to create new environment variables or update existing ones. A child process is created using `fork()` and successfully accesses the exported variable through the process environment. Invalid variable names and incorrect export syntax are detected and handled appropriately.